

Politecnico di Milano

Anno accademico 2013-2014

Ingegneria del Software – Appello del 9 settembre 2014

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri il seguente metodo statico:

```
private static ArrayList<String> tagliaLista(ArrayList<String> l, int n)
```

Il metodo prende in ingresso una lista `l` di Stringhe e un intero `n` e restituisce un'altra lista. La nuova lista deve contenere tutte le stringhe contenute in `l` di lunghezza non inferiore a `n` ordinandole dalla più lunga alla più corta. Se nessuna stringa in `l` dovesse avere lunghezza almeno pari a `n`, il metodo deve comunque restituire un `ArrayList` vuoto.

Si definisca:

1. La pre-condizione sui parametri. Si noti che il metodo non lancia eccezioni.

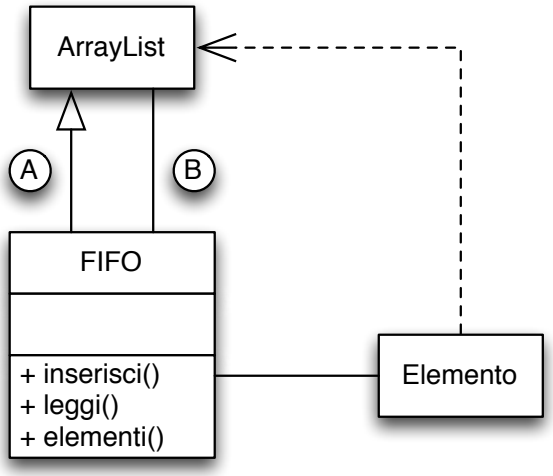
2. La post-condizione relativa alle stringhe contenute nel risultato

3. La post-condizione relativa all'ordinamento delle stringhe nel risultato

4. Ogni altra condizione da aggiungere alla post-condizione e non definita sopra

Esercizio 2

Si consideri il seguente diagramma (schematico e parziale) delle classi UML, che mostra una classe `FIFO` che realizza una struttura dati con politica di accesso FIFO (ossia, una coda):



Il diagramma illustra due possibili alternative, denotate con (A): ereditarietà e (B): associazione per la relazione fra la classe FIFO e la classe `ArrayList` di Java.

1. Quale delle due alternative (A o B) usereste per implementare la classe FIFO attraverso un `ArrayList` Java? Giustificare la risposta.

2. Scrivere un possible rep della classe `FIFO` e spiegare come implementare il metodo `void inserisci(Elemento e)`.

3. Implementare il metodo iteratore `public Iterator<Elemento> elementi()`, assieme a tutte le classi necessarie. Il metodo consente di ottenere tutti gli elementi di una `FIFO`, secondo l'ordine di accesso FIFO.

Esercizio 3

Si consideri un'applicazione software che gestisce gli ordini e i conti di una pizzeria. La pizzeria prevede tre tipi base di pizza, denominati Margherita, Pomodoro, Focaccia. Ogni pizza è associata a una descrizione (di tipo stringa) e ha un proprio prezzo per la versione base. Il cliente può aggiungere a piacere alla pizza base diversi tipi di supplemento, che supponiamo essere Prosciutto, Funghi, Carciofi, Bufala. Ciascun supplemento ha un prezzo e una descrizione. Il prezzo finale di una pizza è calcolato sommando al prezzo base il prezzo di ciascun condimento. Ad esempio, se il prezzo di una Margherita fosse quattro euro, il costo del Prosciutto due euro e quello dei Funghi un euro, il costo di una Pizza Prosciutto e Funghi sarebbe di sette euro.

Si richiede di descrivere una struttura di classi adatta a risolvere il problema, tramite ad esempio un diagramma UML e, se necessario, qualche frammento di codice Java. In particolare, la soluzione prospettata deve consentire di:

- aggiungere combinazioni a piacere dei supplementi;
- calcolare correttamente il prezzo finale di una pizza;
- essere facilmente estendibile nel caso in cui si introducano nuovi supplementi o nuovi tipi di pizza base.

Esercizio 4

Si consideri il metodo Java seguente:

```
public static int foo(int n) {
   int t = n%3;

   while (n != 0)
      if (t == 3) return 3;
      else t = 2;
   return t;
}
```

e si definisca

1. Il diagramma del flusso di controllo
2. Un insieme minimo di test che copra tutte le istruzioni. Nel caso non fosse possibile, definire la copertura (in percentuale) dell'insieme di test definiti.
3. Quali (probabili) errori sono evidenziati dall'insieme di test definito al punto precedente.